

### Programming Assignment 3

Nathan Lenahan

10/1/26

#### Analysis of the two GenAI Programs:

- a) The two GenAI programs used to generate the two programs were Claude and Gemini
- b) Both Claude and Gemini were accessed through their respective websites using Google Chrome

#### c) The following prompt was used for both GenAI programs:

CREATE THE FOLLOWING PROGRAM USING C++ CODE. YOU MUST USE OBJECTS, NOT FUNCTIONS, TO IMPLEMENT THE FOLLOWING REQUIREMENTS.

#### OVERVIEW:

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.

#### TEST FILE FORMAT:

Command: EMAIL <sender category>,<subject line>,<date>

Description of EMAIL command:

The emails in the CEO's Inbox should be placed in a queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

Command: NEXT

Description of NEXT command:

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

Command: READ

Description of READ command:

CEO has read email and dealt with it

Command: COUNT

Description of COUNT command:

Display current count of unread emails

The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

SAMPLE TEST FILE:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

THE SAMPLE TEST FILE PROGRAM SHOULD DISPLAY THIS:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

**d) The following is the generated code from both GenAI programs:**

**CLAUDE - Raw Generated Code**

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// -----
// Email: one message in the CEO's inbox
// -----
class Email {
public:
    string sender;
    string subject;
    string date;    // MM-DD-YYYY
    int senderRank; // higher = read sooner
    int dateKey;    // YYYYMMDD, larger = newer
    int sequence;   // arrival order, used only to break exact ties

    Email() : sender(""), subject(""), date(""),
              senderRank(0), dateKey(0), sequence(0) {}

    Email(const string& s, const string& subj, const string& d, int seq)
        : sender(s), subject(subj), date(d), sequence(seq) {
        senderRank = rankFor(s);
        dateKey = keyFor(d);
    }

    // Returns true if this email should be read before 'other'
    bool hasHigherPriorityThan(const Email& other) const {
        if (senderRank != other.senderRank) {
            return senderRank > other.senderRank;
        }
        if (dateKey != other.dateKey) {
            return dateKey > other.dateKey; // newest first
        }
        return sequence < other.sequence; // earlier arrival first
    }
};
```

```

    }

private:
    static int rankFor(const string& s) {
        if (s == "Boss")        return 5;
        if (s == "Subordinate")  return 4;
        if (s == "Peer")        return 3;
        if (s == "ImportantPerson") return 2;
        return 1;                // OtherPerson
    }

    static int keyFor(const string& d) {
        // MM-DD-YYYY -> YYYYMMDD
        int month = stoi(d.substr(0, 2));
        int day   = stoi(d.substr(3, 2));
        int year  = stoi(d.substr(6, 4));
        return year * 10000 + month * 100 + day;
    }
};

// -----
// MaxHeap: list (array) based, grows as needed
// -----
class MaxHeap {
private:
    Email* items;
    int count;
    int capacity;

    int parentOf(int i) const { return (i - 1) / 2; }
    int leftOf(int i)  const { return 2 * i + 1; }
    int rightOf(int i) const { return 2 * i + 2; }

    void swapItems(int a, int b) {
        Email temp = items[a];
        items[a] = items[b];
        items[b] = temp;
    }

    void grow() {
        int newCapacity = capacity * 2;
        Email* bigger = new Email[newCapacity];
        for (int i = 0; i < count; i++) {
            bigger[i] = items[i];
        }
        delete[] items;
        items = bigger;
        capacity = newCapacity;
    }
}

```

```

void siftUp(int i) {
    while (i > 0) {
        int parent = parentOf(i);
        if (items[i].hasHigherPriorityThan(items[parent])) {
            swapItems(i, parent);
            i = parent;
        } else {
            break;
        }
    }
}

```

```

void siftDown(int i) {
    while (true) {
        int left = leftOf(i);
        int right = rightOf(i);
        int largest = i;

        if (left < count && items[left].hasHigherPriorityThan(items[largest])) {
            largest = left;
        }
        if (right < count && items[right].hasHigherPriorityThan(items[largest])) {
            largest = right;
        }

        if (largest == i) {
            break;
        }
        swapItems(i, largest);
        i = largest;
    }
}

```

public:

```

MaxHeap() : count(0), capacity(16) {
    items = new Email[capacity];
}

```

```

~MaxHeap() {
    delete[] items;
}

```

```

// Heap owns raw memory; disallow copying
MaxHeap(const MaxHeap&) = delete;
MaxHeap& operator=(const MaxHeap&) = delete;

```

```

bool isEmpty() const { return count == 0; }
int size() const { return count; }

```

```

void insert(const Email& e) {
    if (count == capacity) {
        grow();
    }
    items[count] = e;
    siftUp(count);
    count++;
}

const Email& peekMax() const {
    return items[0];
}

void removeMax() {
    if (count == 0) {
        return;
    }
    count--;
    items[0] = items[count];
    siftDown(0);
}
};

// -----
// EmailQueue: the CEO's inbox (priority queue backed by MaxHeap)
// -----
class EmailQueue {
private:
    MaxHeap heap;
    int nextSequence;

public:
    EmailQueue() : nextSequence(0) {}

    void addEmail(const string& sender, const string& subject, const string& date) {
        Email e(sender, subject, date, nextSequence);
        nextSequence++;
        heap.insert(e);
    }

    void showNext() const {
        if (heap.isEmpty()) {
            cout << "There are no emails to read." << endl << endl;
            return;
        }
        const Email& e = heap.peekMax();
        cout << "Next email:" << endl;
        cout << "\tSender: " << e.sender << endl;
    }
};

```

```

        cout << "\tSubject: " << e.subject << endl;
        cout << "\tDate: " << e.date << endl << endl;
    }

    void markRead() {
        heap.removeMax(); // does nothing if empty
    }

    void showCount() const {
        cout << "There are " << heap.size() << " emails to read." << endl << endl;
    }
};

// -----
// CommandProcessor: reads the test file and runs each command
// -----
class CommandProcessor {
private:
    EmailQueue inbox;

    static string trim(const string& s) {
        size_t start = 0;
        size_t end = s.size();
        while (start < end && (s[start] == ' ' || s[start] == '\t' ||
                               s[start] == '\r' || s[start] == '\n')) {
            start++;
        }
        while (end > start && (s[end - 1] == ' ' || s[end - 1] == '\t' ||
                               s[end - 1] == '\r' || s[end - 1] == '\n')) {
            end--;
        }
        return s.substr(start, end - start);
    }

    void handleEmail(const string& args) {
        size_t firstComma = args.find(',');
        size_t lastComma = args.rfind(',');
        if (firstComma == string::npos || firstComma == lastComma) {
            return; // malformed line
        }
        string sender = trim(args.substr(0, firstComma));
        string subject = trim(args.substr(firstComma + 1, lastComma - firstComma - 1));
        string date = trim(args.substr(lastComma + 1));
        inbox.addEmail(sender, subject, date);
    }

public:
    void processFile(const string& filename) {
        ifstream in(filename);
    }
};

```

```

    if (!in) {
        cout << "Could not open file: " << filename << endl;
        return;
    }

    string line;
    while (getline(in, line)) {
        line = trim(line);
        if (line.empty()) {
            continue;
        }

        if (line.compare(0, 6, "EMAIL ") == 0) {
            handleEmail(line.substr(6));
        } else if (line == "NEXT") {
            inbox.showNext();
        } else if (line == "READ") {
            inbox.markRead();
        } else if (line == "COUNT") {
            inbox.showCount();
        }
    }
}

};

int main(int argc, char* argv[]) {
    string filename;
    if (argc > 1) {
        filename = argv[1];
    } else {
        cout << "Enter test file name: ";
        getline(cin, filename);
    }

    CommandProcessor processor;
    processor.processFile(filename);
    return 0;
}

```

### **GEMINI - Raw Generated Code**

```

#include <iostream>
#include <string>
#include <sstream>
#include <vector>

// Map sender category string to integer priority level
int getSenderPriority(const std::string& sender) {
    if (sender == "Boss") return 5;
    if (sender == "Subordinate") return 4;
}

```



```

    if (sender == "Peer") return 3;
    if (sender == "ImportantPerson") return 2;
    if (sender == "OtherPerson") return 1;
    return 0;
}

// Helper to convert MM-DD-YYYY string to YYYYMMDD string for chronological comparison
std::string parseDateToComparable(const std::string& dateStr) {
    if (dateStr.length() < 10) return "";
    std::string month = dateStr.substr(0, 2);
    std::string day = dateStr.substr(3, 2);
    std::string year = dateStr.substr(6, 4);
    return year + month + day;
}

// Email Class Object
class Email {
private:
    std::string sender;
    std::string subject;
    std::string date;

public:
    Email() : sender(""), subject(""), date("") {}
    Email(std::string s, std::string subj, std::string d)
        : sender(s), subject(subj), date(d) {}

    std::string getSender() const { return sender; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return date; }

    // Priority Comparison:
    // 1. Higher sender priority comes first.
    // 2. If same priority, newer email (lexicographically greater YYYYMMDD) comes first.
    bool operator<(const Email& other) const {
        int p1 = getSenderPriority(this->sender);
        int p2 = getSenderPriority(other.sender);

        if (p1 != p2) {
            return p1 < p2;
        }

        std::string d1 = parseDateToComparable(this->date);
        std::string d2 = parseDateToComparable(other.date);

        return d1 < d2;
    }

    bool operator>(const Email& other) const {

```

```

        return other < *this;
    }
};

// Custom MaxHeap Implementation (List/Array-based)
class MaxHeap {
private:
    std::vector<Email> heap;

    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (heap[parent] < heap[index]) {
                std::swap(heap[parent], heap[index]);
                index = parent;
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        int size = heap.size();
        while (true) {
            int left = 2 * index + 1;
            int right = 2 * index + 2;
            int largest = index;

            if (left < size && heap[largest] < heap[left]) {
                largest = left;
            }
            if (right < size && heap[largest] < heap[right]) {
                largest = right;
            }

            if (largest != index) {
                std::swap(heap[index], heap[largest]);
                index = largest;
            } else {
                break;
            }
        }
    }

public:
    MaxHeap() {}

    void insert(const Email& email) {
        heap.push_back(email);
    }

```

```

        heapifyUp(heap.size() - 1);
    }

    bool isEmpty() const {
        return heap.empty();
    }

    Email peek() const {
        if (!isEmpty()) {
            return heap[0];
        }
        return Email();
    }

    Email extractMax() {
        if (isEmpty()) {
            return Email();
        }
        Email root = heap[0];
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) {
            heapifyDown(0);
        }
        return root;
    }

    int count() const {
        return heap.size();
    }
};

// PriorityQueue Object wrapper over MaxHeap
class EmailPriorityQueue {
private:
    MaxHeap maxHeap;

public:
    void addEmail(const Email& email) {
        maxHeap.insert(email);
    }

    void displayNext() const {
        if (maxHeap.isEmpty()) {
            std::cout << "No emails in queue." << std::endl;
            return;
        }
        Email top = maxHeap.peek();
        std::cout << "Next email:\n";
    }
};

```

```

        std::cout << "\tSender: " << top.getSender() << "\n";
        std::cout << "\tSubject: " << top.getSubject() << "\n";
        std::cout << "\tDate: " << top.getDate() << "\n\n";
    }

    void readNext() {
        if (!maxHeap.isEmpty()) {
            maxHeap.extractMax();
        }
    }

    void displayCount() const {
        int cnt = maxHeap.count();
        std::cout << "There are " << cnt << " emails to read.\n\n";
    }
};

// Main Command Processor
int main() {
    EmailPriorityQueue emailSystem;
    std::string line;

    while (std::getline(std::cin, line)) {
        if (line.empty()) continue;

        if (line == "COUNT") {
            emailSystem.displayCount();
        } else if (line == "NEXT") {
            emailSystem.displayNext();
        } else if (line == "READ") {
            emailSystem.readNext();
        } else if (line.rfind("EMAIL ", 0) == 0) {
            std::string payload = line.substr(6); // Remove "EMAIL "
            std::stringstream ss(payload);
            std::string sender, subject, date;

            if (std::getline(ss, sender, ',') &&
                std::getline(ss, subject, ',') &&
                std::getline(ss, date, ',')) {

                emailSystem.addEmail(Email(sender, subject, date));
            }
        }
    }

    return 0;
}

```

e) **The following is the analysis and comparison of the two GenAI programs:**

***Correctness:***

Both Claude and Gemini compiled and ran correctly during testing. They each prioritize emails according to the required sender categories and dates. Both programs also passed the sample test file as well as additional test files I created for an empty queue, repeated NEXT and READ commands, and different sender priorities and dates. Both correctly kept the appropriate email on top after repeated NEXT commands and removed the highest-priority email after READ. This also confirmed that both programs handle the special command sequences mentioned in the assignment instructions.

Claude seemed to do a better job of using classes than Gemini did. Gemini used three classes, Email, MaxHeap, and EmailPriorityQueue, but also created separate functions like getSenderPriority and parseDateToComparable outside of the classes. Claude organized almost all of the program inside objects using the Email, MaxHeap, EmailQueue, and CommandProcessor classes. I think this is an important thing to keep in mind since the instructions explicitly said to use objects, not functions.

There is also a difference in the way that tie-breaking is handled. Claude stores a sequence value for each email. If two emails have the same priority and the same date, Claude uses the arrival order as the final tie-breaker. Gemini compares priority and then date but doesn't have a final tie-breaker as Claude does. This doesn't make Gemini wrong, but Claude handles that situation slightly better.

Both programs also handle the input a bit differently. Gemini reads commands directly from standard input, while Claude uses a filename and opens the file. Neither method is necessarily incorrect since the assignment states a test file will be used, but it does not specify exactly how the test file must be provided to the program. This is still an important difference to keep in mind since supporting more than one input method could make the program easier to test and use.

***Execution Time:***

Both programs' execution time is very similar due to the fact that each uses a MaxHeap. Inserting an email takes  $O(\log n)$  time because the new email may have to be moved up the heap to get to its correct spot. Claude uses siftUp, while Gemini uses heapifyUp.

Removing the highest priority email also takes  $O(\log n)$  time since there's a chance that the heap has to be reorganized if the root gets removed. NEXT takes  $O(1)$  time because the highest priority email is already at the top, so it doesn't have to navigate the heap. COUNT also takes  $O(1)$  time since both programs also keep track of the current count of emails.

A difference that can be related to execution time is the way the programs differ when handling resizing memory storage. Claude manually doubles the capacity of its array when it is full; Gemini uses a vector that resizes automatically when needed. A resize can take  $O(n)$  time

temporarily because stored emails may need to be copied into a larger block of memory. This doesn't happen on every email insertion, though, so it doesn't have a huge effect on overall performance.

### ***Space Complexity:***

Similar to execution time, both programs share the same space complexity of  $O(n)$  since the amount of memory needed goes up based on the number of emails stored in the heap. Claude uses a dynamically allocated array to store emails, increasing the capacity when it is needed. Gemini stores the emails in a vector, which grows on its own as emails are added to it.

Gemini's memory management is simpler due to the fact that the resizing of memory is handled automatically by the vector. Claude requires more effort and code to allocate, resize, and free memory, but both programs still have an overall space complexity of  $O(n)$ . Claude starts with a capacity of 16 and includes a destructor to free the allocated array when the program is done running. Neither program uses another major data structure that grows with the number of emails.

### ***Maintainability:***

Claude's code is more object-oriented than Gemini is and does a better job separating the program into separate classes with their own purposes. The Email class stores and compares email information, MaxHeap manages the heap, EmailQueue manages the priority queue, and CommandProcessor handles the input commands. This makes readability easier, and it is easier to tell which part of the program is doing what. Claude also uses a clear `hasHigherPriorityThan` method for comparing, while Gemini overloads the comparison operators. Even though Gemini's is shorter, Claude's is overall easier to understand.

Gemini is shorter and has simpler memory management with the vector implementation. The downside of Gemini is that it isn't as object-oriented as Claude's code. Some logic is stored in functions that are outside of the classes, and commands are handled directly in main. This fact alone makes Gemini less object-oriented than Claude, which is an important distinction considering the program requirements. Claude overall followed the object-oriented requirement better than Gemini, but Gemini does have simpler memory management and a shorter overall implementation. Both programs already have some built-in comments, which is a good plus.

### **f) Program selection and justification: Claude**

I chose Claude to work with for the assignment. The main reason is that Claude followed the object-oriented requirement more closely than Gemini did. Claude did better at making respective classes for specific tasks; while Gemini did that too, it didn't use it to the same extent that Claude did. Gemini still had some standalone functions like `getSenderPriority` and `parseDateToComparable`. Claude also has the extra sequence functionality to handle tiebreakers more predictably. I also thought the way Claude was organized was easier to read. The Email, MaxHeap, EmailQueue, and CommandProcessor classes have a clear purpose that contributes to the program. Overall, Claude seemed like the better option since it matched the requirements more closely and had a structure that seemed easier to improve upon.

**g) The following is a summary of the improvements made to the code**

***Correctness:***

Several changes were made to improve the correctness and reliability of the provided code by Claude. Firstly, I revised the input handling so the program can either use a test file provided through the command line or read commands from standard input. This allows the program to support both `./assignment3 test.txt` and `./assignment3 < test.txt`, making it more flexible for different testing methods. I also changed `peekMax()` to return `nullptr` when the heap is empty instead of just assuming that an email exists. `showNext()` now checks for this before attempting to display an email, which prevents the program from treating `items[0]` as a valid email when the heap is empty. I also changed `markRead()` so that using `READ` when there are no emails displays “No emails to read.” instead of silently doing nothing. Finally, I changed the output for `COUNT` so it displays “There is 1 email to read”, keeping proper grammar when only one email is left. Overall, these changes improve the reliability of the program without affecting its required behavior.

***Execution Time:***

The main time complexities of the program stayed the same since the provided code from Claude already implemented the `MaxHeap` efficiently. Adding an email still takes  $O(\log n)$ , removing an email with `READ` still takes  $O(\log n)$ , `NEXT` is  $O(1)$ , and `COUNT` is  $O(1)$ . Some smaller efficiency improvements were made. In `removeMax()`, the program no longer copies the last heap element and calls `siftDown()` when the final email is removed. I also replaced `endl` with newline characters so the program doesn’t unnecessarily flush the output stream after every line. In `showCount()`, I stored the result of `heap.size()` in a local variable instead of repeatedly calling the method to get its value. These changes do not alter the Big-O complexities, but they reduce unnecessary assignments, heap operations, function calls, and output-stream flushes.

***Space Complexity:***

The overall space complexity also remained  $O(n)$  after revisions were made, where  $n$  is the number of unread emails stored in the `MaxHeap`. I didn’t add any additional data structures that grow with the number of emails, so the improvements that were made didn’t increase the space complexity. Supporting both input methods may require an `ifstream` object when a filename is provided, but this only uses a constant amount of additional space and does not change the overall  $O(n)$  space complexity. I also made the new getter methods return `const string&`, which allows the program to access the stored strings without creating extra copies of the strings. The heap still uses a dynamically allocated array that grows as needed, so its memory usage remains proportional to the number of emails and the overall space complexity remains  $O(n)$ .

### ***Maintainability:***

Several changes were made to improve the organization and maintainability of the code. The data members in the Email class were changed from public to private, and read-only getter methods were added. This improves encapsulation because it stops other classes from being able to modify information in the Email class and only allows it to be read. I also removed the unused isEmpty() method after peekMax() was changed to directly handle the empty-heap case. This removes unnecessary code that went unused and avoids having two different ways to check for the same condition. I also thoroughly added comments to the program and added source attribution so the source of the code is clearly listed within the program itself. I also revised processInput() so that it accepts an input stream to allow the code to work with either a file or standard input. Usage instructions were also added to show how to run the program. Overall, the code maintains the object-oriented structure but has clearer class responsibilities and safer access to data.

**Other GenAI Assistance:** ChatGPT was used for assistance with prompting, analyzing the generated programs, making improvements, and adding comments.